

Lab 1 – Mastering AI for EEE & Programming**Objective**

- ◆ To transition from "Answer-Seeking" to "Concept-Mastery" using Generative AI.

Introduction / Briefing

- ◆ Instructions: Do not use the AI as a calculator. Your goal is to use the "Three Layers of a Great Prompt" (Persona, Constraint, Task) to deepen your understanding of logic and physical constraints.
 - ◆ Warning: If the AI gives you the direct answer or final code immediately, you have failed the prompt constraint.
-
- ◆ To do this lab, use Gen AI tool that is available.

Exercise 1: The "Conceptual Bridge" (EEE Focus)

Goal: Understand the *why* behind physical laws rather than just the formula.

- **Scenario:** You are studying **Kirchhoff's Voltage Law (KVL)**. Instead of asking AI to solve a circuit, you want to understand its relationship to the Law of Conservation of Energy.
- **Sample Prompt:** "I am studying KVL. Do not solve a circuit for me. Instead, explain the relationship between KVL and the Law of Conservation of Energy using a water-pressure analogy. Then, ask me a conceptual question to see if I understand how energy is 'lost' in a resistor." **Try it.**

Step 1: Select a core EEE formula (e.g., KVL, $V=IR$) and write it below.

Step 2: Construct a prompt.

- **Requirement:** Ask the AI to explain the physical intuition using an analogy (e.g., water pressure).
- **Constraint:** Explicitly state: "**Do not solve a circuit for me.**"
- **Task:** Command the AI to end its response with a conceptual question for *you* to answer.

Your Prompt:

Your Output: Write down the answer you gave to the AI's conceptual question regarding energy loss or circuit behavior.

Exercise 2: The "Code Logic Architect" (Programming Focus)

Goal: Improve algorithmic thinking without seeing the final code.

- **Scenario:** You need to write a C++ script to sort a list of sensor data, but you are stuck on the logic.

- **Sample Prompt:**

"I need to write a C++ function to find the average of only the positive numbers in a list. Do not write any code. Provide a **pseudocode outline** or a flow of logic steps. After that, give me a 'buggy' hint of a common mistake students make with loop counters so I can avoid it." **Try it.**

Step 1: Describe a programming goal (e.g., sorting a list, finding averages).

Step 2: Construct a prompt.

- *Constraint:* **Explicitly forbid the AI from providing a code block.**
- *Task:* Ask for a pseudocode outline, a logic flowchart, or a "buggy hint" regarding common mistakes (like loop counters).

Your Prompt:

AI Response Summary: Copy the specific "buggy hint" or logic step the AI warned you about.

Exercise 3: The "Circuit Troubleshooter" (EEE Focus)

Goal: Develop diagnostic skills by analyzing failure states.

- **Scenario:** You have a theoretical circuit, and the output voltage is zero.
- **Sample Prompt:**

"In a series circuit with a 12V battery and two resistors, the measured voltage across the second resistor is 0V. Give me three different physical 'failure' hypotheses (e.g., short circuit, open circuit). For each hypothesis, explain how I would use a multimeter to verify it." **Try it.**

Step 1: Define a "broken" state in a system (e.g., "The voltage across the second resistor is 0V").

Step 2: Construct a prompt.

- *Task:* Ask the AI for three physical "failure" hypotheses (e.g., open circuit vs. short circuit).
- *Task:* Ask how to use a multimeter to verify each hypothesis.

Your Prompt:

Diagnostic Plan: List one verification method (e.g., multimeter setting and placement) suggested by the AI.

Exercise 4: The "Edge-Case Stress Test" (Programming Focus)

Goal: Learn to anticipate errors and improve code robustness.

- **Scenario:** You've written a basic calculator program in C++.
- **Sample Prompt:**

"I have a function that takes two integers and divides them. I want to improve my 'defensive programming' skills. List 5 'edge cases' or 'malicious inputs' that could crash this function or produce logical errors. Explain why each one is a threat." **Try it.**

Step 1: Paste a brief function description or logic idea below.

Step 2: Construct a prompt.

- *Persona:* Ask the AI to act as a **Quality Assurance (QA) Engineer**.
- *Task:* Request a list of 5 "malicious inputs" or edge cases (e.g., Null values, Overflow) that could crash the system.

Your Prompt:

Risk Analysis: List two specific edge cases the AI identified that you had not previously considered.

1.

2.

Exercise 5: The "Reverse Instructor" (Comprehensive)

Goal: Test your own knowledge through peer-review.

- **Scenario:** You want to check if your summary of **Thevenin's Theorem** is correct.
- **Sample Prompt:**

"I am going to explain Thevenin's Theorem to you as if I am the teacher. I want you to act as a student who is confused. Critically analyze my explanation for any technical inaccuracies or 'missing links' in logic. Tell me where my explanation is weak." **Try it.**

Step 1: Choose a complex topic (e.g., Understand C++ pointers) and write a brief explanation of it below in your own words.

Step 2: Construct a prompt.

- *Persona:* Tell the AI to act as a **student who is confused**.
- *Task:* Ask the AI to critically analyze your explanation for "missing links" or weak logic.

Your Prompt:

Critique & Refinement: What specific weakness did the AI find in your explanation? How did you correct it?

Reflection Question for Students:

How did your understanding of the topic change when you were forbidden from asking the AI for the direct answer?

The AI Study Mentor: Power-Prompt Cheat Sheet

1. The "Physical Intuition" Template (EEE)

Use when: You know the formula but don't understand the physical behavior.

"Explain the concept of **[Insert Topic, e.g., Impedance or Capacitance]** without using any math first. Use a real-world physical analogy (like water, traffic, or heat). Once I confirm I understand the analogy, show me how the standard formula **[Insert Formula]** represents that physical behavior."

2. The "Logic First" Template (Programming)

Use when: You have a coding task but don't want to skip the thinking process.

"I am working on a **[Language, e.g., Python/C++]** program to **[Insert Task]**. Before I write any code, act as a lead developer. Provide a step-by-step logic flowchart in text form. Identify where I should use **[Insert Concept, e.g., nested loops or recursion]** and explain why that is the most efficient choice."

5. The "Socratic Exam Prep" (Comprehensive)

Use when: You think you know a topic and want to be tested.

"I am studying **[Insert Module Topic]**. Act as a professor. Ask me one challenging 'high-order' conceptual question that requires me to apply **[Concept A]** to **[Concept B]**. Once I answer, critique my technical accuracy and tell me if I missed any critical engineering constraints."

Why this works:

- **Active Recall:** It forces you to retrieve information from your own brain.
- **Deep Encoding:** It connects abstract programming concepts to logical structures.
- **Verification:** It mimics the "Human in the Loop" approach used in professional AI development.

~ The END ~